

Working with databases in python3

Databases in Python

- Databases store and manage large amounts of structured or unstructured data.
- Python supports SQL and NoSQL databases through various libraries.
- Databases provide efficient storage, retrieval, and updating of data.
- SQL databases organize data into tables, while NoSQL databases use flexible structures.
- Python can connect to local or remote databases.
- Database skills are essential for Data Science, Backend Development, and Analytics.

SQLite Database

- SQLite is a lightweight, file-based relational database.
- It does not require a separate database server.
- Data is stored in a local file.
- SQLite is ideal for development, testing, and small applications.
- Python includes built-in SQLite support through the sqlite3 module.
- It is commonly used for prototypes and standalone applications.

Example

```
import sqlite3
```

```
database = sqlite3.connect("portfolio.db")  
cursor = database.cursor()
```

Creating Tables in SQLite

- Tables organize data into rows and columns.
- Each column stores a specific type of information.
- SQL commands are used to create tables.
- Tables are created only once.
- SQLite stores tables inside the database file.
- Well-designed tables improve data management.

Example

```
create_table_query = """
CREATE TABLE investments (
    coin_id TEXT,
    currency TEXT,
    sell INT,
    amount REAL,
    date TIMESTAMP
);
"""
```

```
cursor.execute(create_table_query)
```

Inserting Data into SQLite

- Records are inserted using SQL INSERT statements.
- Parameterized queries improve security.
- Parameters prevent SQL injection attacks.
- Data is stored as rows in tables.
- Changes must be committed to the database.
- Insert operations add new information.

Example

```
import datetime
```

```
investment = (
```

```
"bitcoin",
"usd",
True,
1.0,
datetime.datetime.now()
)

cursor.execute(
    "INSERT INTO investments VALUES (?, ?, ?, ?, ?);",
    investment
)

database.commit()
```

Retrieving Data from SQLite

- SQL SELECT statements retrieve data.
- Results are returned as rows.
- Multiple rows can be fetched together.
- Single rows can also be retrieved.
- Retrieved data can be processed in Python.
- Querying is one of the most common database operations.

Example

```
result = cursor.execute(
    "SELECT * FROM investments;"
)

all_rows = result.fetchall()
```

Fetch Methods

- `fetchall()` retrieves all matching rows.
- `fetchone()` retrieves only the first row.

- Results are returned after query execution.
- Fetch methods simplify data retrieval.
- Retrieved rows can be iterated and processed.
- They are part of the database cursor API.

Example

```
all_rows = result.fetchall()
```

```
first_row = result.fetchone()
```

Row Factories

- Row factories transform query results into more useful objects.
- Default SQLite results are tuples.
- Named access is easier than positional indexing.
- They improve readability.
- Custom row factories can create Python objects.
- They simplify larger applications.

Example

```
database.row_factory = investment_row_factory
```

Dataclasses with SQLite

- Dataclasses simplify object creation.
- They automatically generate constructors.
- Database rows can be converted into dataclass objects.
- They improve readability.
- Dataclasses reduce boilerplate code.
- They work well with row factories.

Example

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Investment:
```

```
    coin_id: str
```

```
    currency: str
```

```
    amount: float
```

```
    sell: bool
```

```
    date: datetime.datetime
```

PostgreSQL

- PostgreSQL is a powerful relational database server.
- Unlike SQLite, it runs as a separate service.
- Applications connect through a network.
- Authentication is required.
- PostgreSQL supports enterprise applications.
- It is widely used in production environments.

psycopg2

- psycopg2 is the most popular PostgreSQL library for Python.
- It enables communication with PostgreSQL servers.
- SQL queries can be executed directly.
- Results are returned to Python programs.
- It supports parameterized queries.
- The psycopg2-binary package simplifies installation.

Example

```
import psycopg2
```

```
connection = psycopg2.connect(
```

```
host="localhost",  
database="manager",  
user="postgres",  
password="pgpassword"  
)
```

Querying PostgreSQL

- Queries are executed through cursors.
- SQL syntax remains the same.
- Results can be fetched into Python.
- Connections should be closed after use.
- Cursors manage database interactions.
- Query execution follows the connect-execute-fetch pattern.

Example

```
cursor = connection.cursor()
```

```
cursor.execute(  
    "SELECT * FROM investment;"  
)
```

```
data = cursor.fetchall()
```

```
cursor.close()  
connection.close()
```

NoSQL Databases

- NoSQL databases store data differently from relational databases.
- They complement SQL databases rather than replace them.
- Data is often stored as documents.
- NoSQL systems provide greater flexibility.

- Schemas are often optional.
- Popular examples include MongoDB, Redis, Cassandra, and Neo4j.

MongoDB

- A database contains collections.
- Collections contain documents.
- Documents are similar to JSON objects.
- Documents do not require fixed schemas.
- Different documents may have different structures.
- MongoDB is one of the most popular NoSQL databases.

Example

```
{  
  "coin_id": "bitcoin",  
  "amount": 1.0  
}
```

Mongita

- Mongita is an embedded NoSQL database.
- It is conceptually similar to SQLite.
- It implements part of the MongoDB API.
- It stores document-based data locally.
- It is useful for testing and development.
- It uses Python dictionaries as documents.

Example

```
from mongita import MongitaClientDisk
```

```
client = MongitaClientDisk()
```

```
db = client.portfolio
```

```
investments = db.investments
```

CRUD Operations in Mongita

- CRUD stands for Create, Read, Update, and Delete.
- Documents are inserted using `insert_one()`.
- Documents are retrieved using `find()`.
- Documents are updated using `update_many()`.
- Documents are deleted using `delete_one()`.
- CRUD forms the foundation of database interaction.

Example

```
investments.insert_one({  
    "coin_id": "bitcoin",  
    "currency": "usd",  
    "amount": 1.0  
})
```

MongoDB with pymongo

- pymongo is the official MongoDB Python library.
- It provides full MongoDB functionality.
- It supports filtering and updates.
- Embedded documents are supported.
- Array operations are supported.
- It is used for production MongoDB applications.

Example

```
from pymongo import MongoClient
```

```
client = MongoClient()
```

```
db = client.portfolio
```

```
investments = db.investments
```

Filtering Documents

- Filters specify which documents should be returned.
- Multiple conditions can be combined.
- MongoDB uses JSON-like query syntax.
- \$and combines multiple filters.
- Filtering improves query precision.
- Queries can target nested fields.

Example

```
{
  "$and": [
    {"coin": "bitcoin"},
    {"amount": {"$gt": 2}}
  ]
}
```

Embedded Documents

- Documents can contain nested documents.
- Nested structures improve organization.
- Related information stays together.
- Embedded documents reduce joins.
- Dot notation accesses nested fields.
- They are widely used in MongoDB.

Example

```
watchlist = {
  "name": "My Watchlist",
  "metadata": {
    "currency": "USD",
```

```
    "description": "This is a watchlist"
  }
}
```

Embedded Arrays

- Arrays store multiple embedded documents.
- Related records can be grouped together.
- Arrays support flexible structures.
- New elements can be appended easily.
- Array data can be queried.
- MongoDB provides special array operators.

Example

```
watchlist = {
  "name": "My Watchlist",
  "coins": [
    {"coin": "bitcoin"},
    {"coin": "ethereum"}
  ]
}
```

MongoDB Update Operators

- Update operators modify existing documents.
- \$inc increases numeric values.
- \$push adds elements to arrays.
- \$pull removes elements from arrays.
- Operators update data efficiently.
- They avoid replacing entire documents.

Example

```
{
  "$inc": {
    "amount": 1
  }
}
```

```
}  
}
```

SQLAlchemy ORM

- SQLAlchemy is Python's most popular ORM.
 - ORM stands for Object Relational Mapper.
 - It maps Python objects to database tables.
 - SQLAlchemy supports multiple database systems.
 - Developers can work mostly with Python code.
 - It reduces the amount of raw SQL required.
-

ORM Concepts

- ORM creates a bridge between objects and tables.
- Tables map to classes.
- Rows map to objects.
- Columns map to attributes.
- CRUD operations use Python objects.
- ORM improves developer productivity.

Example

```
class Investment:  
    id: int  
    coin: str  
    currency: str  
    amount: float
```

Creating SQLAlchemy Models

- Models represent database tables.
- Each model inherits from a base class.
- Attributes define table columns.

- Type annotations improve readability.
- Models provide structure for data.
- SQLAlchemy automatically generates tables.

Example

```
from sqlalchemy.orm import DeclarativeBase
```

```
class Base(DeclarativeBase):  
    pass
```

```
class Investment(Base):  
    __tablename__ = "investment"
```

Column Mapping

- Model attributes map to table columns.
- Data types are explicitly defined.
- Primary keys uniquely identify records.
- Constraints improve data integrity.
- Mapped columns define database structure.
- SQLAlchemy generates SQL automatically.

Example

```
id = mapped_column(primary_key=True)
```

```
coin = mapped_column(String(32))
```

```
currency = mapped_column(String(3))
```

SQLAlchemy Engine and Session

- The engine manages database connections.
- Sessions manage transactions.
- Sessions perform CRUD operations.
- Sessions track object changes.

- Changes require commits.
- They are central components of SQLAlchemy.

Example

```
from sqlalchemy import create_engine
```

```
engine = create_engine(  
    "sqlite:///data.db"  
)
```

```
from sqlalchemy.orm import Session
```

```
with Session(engine) as session:  
    session.add(bitcoin)  
    session.commit()
```

Querying with SQLAlchemy

- Queries retrieve model objects.
- Filters limit results.
- Query results become Python objects.
- SQLAlchemy generates SQL automatically.
- Object access feels natural.
- Complex queries remain readable.

Example

```
stmt = select(Investment).where(  
    Investment.coin == "bitcoin"  
)
```

```
bitcoin = session.execute(stmt).scalar_one()
```

Relationships

- Relationships connect related tables.
- Foreign keys establish links.
- One-to-many relationships are common.

- Relationships simplify navigation between records.
- SQLAlchemy manages joins automatically.
- Related objects can be accessed directly.

Example

```
portfolio_id = mapped_column(
    ForeignKey("portfolio.id")
)
```

One-to-Many Relationship Example

- One portfolio can contain many investments.
- Each investment belongs to one portfolio.
- Relationships are defined in both classes.
- back_populates keeps objects synchronized.
- Related data becomes easier to access.
- This is one of the most common database patterns.

Example

```
class Portfolio(Base):
```

```
    investments = relationship(
        back_populates="portfolio"
    )
```

```
class Investment(Base):
```

```
    portfolio = relationship(
        back_populates="investments"
    )
```

MongoDB and Mongita (NoSQL Databases)

What is NoSQL?

- NoSQL databases complement SQL databases rather than replace them.
- They provide flexible data storage models.

- Data may be stored as documents, key-value pairs, columns, or graphs.
- Common NoSQL databases include:
 - MongoDB (Document Database)
 - Redis (Key-Value Store)
 - Cassandra (Column-Oriented Database)
 - Neo4j (Graph Database)
- NoSQL databases are often schema-less.
- They are suitable for large-scale and flexible applications.

SQL vs NoSQL

SQL	NoSQL
Table-based	Document/Key-Value/Graph
Fixed Schema	Flexible Schema
Relational	Non-relational
Uses SQL	Uses JSON-like Documents
Example: PostgreSQL	Example: MongoDB